

Contents

Lesson 4: Comments, Booleans, and Conditionals	1
1. Comments	1
2. Booleans → Python's <code>bool</code> Type	1
Comparison Operators	1
Combining Conditions	2
More Examples with <code>not</code> and <code>or</code>	2
A Word of Caution with Floats	2
Test Your Understanding	3
3. Conditionals → Python's <code>if</code> Statement	3
Adding an Alternative: <code>else</code>	3
Checking Several Conditions: <code>elif</code>	3
Summary	4

Lesson 4: Comments, Booleans, and Conditionals

So far, every line of code we've written has run every single time, in order, from top to bottom. In this lesson, we introduce two new ideas: how to leave notes inside your code without affecting it, and how to make Python *decide* what to do based on a condition — much like an “if this, then that” statement in everyday reasoning, or a piecewise definition in mathematics.

1. Comments

Sometimes you want to leave a note in your code — for yourself, or for someone reading it later — without Python trying to run it as an instruction. Python ignores anything after a `#` symbol on a line.

```
# This calculates the area of a circle
radius = 7
area = 3.14159 * radius ** 2
```

The line starting with `#` is completely ignored when the code runs — it's there purely for humans to read. This becomes especially useful once your code involves several steps, since a well-placed comment can explain *why* a line exists, not just what it does.

2. Booleans → Python's `bool` Type

In mathematics, a statement is often either **true** or **false** — for example, “5 is greater than 3” is true, while “5 is less than 3” is false. Python has a dedicated data type for exactly this: `bool`, which can only hold one of two values, `True` or `False`.

`True`

`False`

Notice the capital letters — `true` and `false` (lowercase) are not valid in Python; it must be `True` and `False`.

Comparison Operators

To actually produce a boolean value, we compare things using **comparison operators**. These look similar to the ones you already use in mathematics, with one important difference.

```
5 > 3
```

This gives `True`. Python supports the full set of comparisons:

Mathematical idea	Python
$=$ (is equal to)	<code>==</code>
$\neq$ (is not equal to)	<code>!=</code>
$>$	<code>></code>
$<$	<code><</code>
$\geq$	<code>>=</code>
$\leq$	<code><=</code>

The most important one to notice is equality. In mathematics, we write $5 = 5$ using a single $=$. But recall from Lesson 3: in Python, a single $=$ is the *assignment* operator, not a comparison. To check whether two things are equal, Python uses **two** equals signs, `==`.

```
5 == 5
```

```
5 == 3
```

The first gives **True**, the second gives **False**. Mixing up $=$ and `==` is one of the most common mistakes beginners make — $=$ stores a value, `==` asks a question.

Combining Conditions

Just as in logic, we can combine multiple true/false statements using **and**, **or**, and **not**.

```
x = 7
x > 5 and x < 10
```

This gives **True**, since both conditions hold — x is greater than 5, *and* less than 10. This mirrors the mathematical statement $5 < x < 10$.

More Examples with not and or

```
print(not(2 > 3))
```

Since $2 > 3$ is **False**, **not** flips it to **True**.

```
2 > 3 or 3 > 2
```

This gives **True** — **or** only needs *one* of the two conditions to hold, and $3 > 2$ is true.

We can also check a genuinely mathematical statement — the Pythagorean theorem for a 3-4-5 triangle:

```
3 ** 2 + 4 ** 2 == 5 ** 2
```

This gives **True**, since $3^2 + 4^2 = 25$ and $5^2 = 25$ — confirming the relationship holds (with equality, in this case).

A Word of Caution with Floats

Be careful when working with **float** values — computations are only carried out to a certain precision, since computers store decimals in binary rather than exactly.

```
0.1 + 0.2 == 0.3
```

This gives **False** — which is surprising at first! Let's see why:

```
print(0.1 + 0.2)
```

This actually prints `0.30000000000000004`, not `0.3`. This tiny rounding error comes from how binary floating-point numbers are stored, and it isn't a Python bug — it's true of nearly every programming

language. This becomes especially important later when we work with numerical methods, where small floating-point errors can accumulate.

Test Your Understanding

Is the following statement correct?

```
(1 < 2 < 3) == (1 < 2) and (2 < 3)
```

Running this gives `True` — but the reasoning is subtler than it looks. Python’s `and` has *lower* precedence than `==`, so this is actually evaluated as `((1 < 2 < 3) == (1 < 2)) and (2 < 3)`, not as a direct comparison between the whole chained statement and the whole “and” statement. It happens to still give `True` here, but to compare the two ideas unambiguously, you’d want to write it with explicit parentheses around the entire right-hand side:

```
(1 < 2 < 3) == ((1 < 2) and (2 < 3))
```

This is a good reminder that when in doubt, adding parentheses costs nothing and removes any ambiguity about what Python will actually evaluate first.

3. Conditionals → Python’s `if` Statement

Now we combine booleans with a new structure: the ability to tell Python “only run this code *if* a condition is true.” This is written using `if`.

```
x = 7
if x > 5:
    print("x is greater than 5")
```

Notice two things here: the line ends with a colon `:`, and the line below it is **indented** — pushed in slightly from the left margin. Python uses this indentation to know which lines belong inside the `if` statement. This is different from many other programming languages, and it’s not optional in Python — incorrect indentation causes an error.

Adding an Alternative: `else`

Often, we want to say “do this if the condition is true, otherwise do something else.” This is where `else` comes in.

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

Since `x` is 3, the condition `x > 5` is `False`, so Python skips the first line and runs the `else` block instead, printing “x is not greater than 5”.

Checking Several Conditions: `elif`

Sometimes there’s more than one possibility to check. `elif` (short for “else if”) lets us chain multiple conditions together.

```
x = 5
if x > 5:
    print("x is greater than 5")
elif x == 5:
    print("x is exactly 5")
```

```
else:
    print("x is less than 5")
```

Python checks each condition in order, from top to bottom, and runs the code under the **first** one that is **True**, skipping the rest. Here, since `x == 5` is **True**, Python prints "x is exactly 5" and never even checks the **else**.

This mirrors how a mathematician might define a piecewise function:

$$f(x) = \begin{cases} \text{"greater"} & \text{if } x > 5 \\ \text{"exactly 5"} & \text{if } x = 5 \\ \text{"less"} & \text{if } x < 5 \end{cases}$$

Each **elif** is simply another case in that piecewise definition, checked in order.

Summary

Mathematical idea	Python
A true/false statement	<code>bool (True / False)</code>
<code>=</code> (equality question)	<code>==</code>
<code>≠</code>	<code>!=</code>
A note explaining reasoning	<code># comment</code>
A piecewise condition	<code>if / elif / else</code>

A conditional lets Python choose which instructions to run, based on whether a statement is true or false — just like a piecewise definition chooses which formula applies.